

MODULE 2

Role of the Syntax Analyser – Syntax error handling. Review of Context Free Grammars - Derivation and Parse Trees, Eliminating Ambiguity. Basic parsing approaches - Eliminating left recursion, left factoring. Top-Down Parsing - Recursive Descent parsing, Predictive Parsing, LL(1) Grammars

Role of the Syntax Analyzer/Parser

- The parser obtains a string of tokens from the lexical analyser and verifies that the string of token names can be generated by the grammar for the source language.
- The parser constructs a parse tree and passes it to the rest of the compiler for further processing.
- The parser should report any syntax error and recover from the commonly occurring errors to continue processing the remainder of the program.

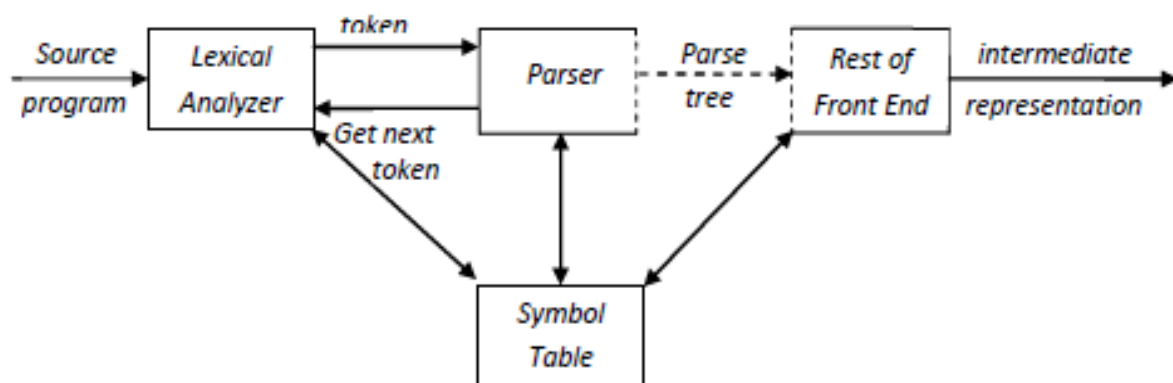


Figure 3.1: Position of parser in compiler model

Position of parser in compiler model

Syntax Error Handling

The error handler in a parser has the following goals:

- It should report the presence of errors clearly and accurately.
- It should recover from each error quickly enough to be able to detect subsequent errors.
- It should not significantly slow down the processing of correct program.

Error Recovery Strategies:

1. Panic Mode Recovery
2. Phrase-Level Recovery
3. Error Productions
4. Global Correction

Panic Mode Recovery

- Simplest method used by most parsing methods.
- On discovering an error the parser discards input symbols one at a time until one of a set of synchronizing tokens is found.
- Synchronising tokens are usually delimiters such as semicolon, } etc.
- It is guaranteed not to go into an infinite loop.

Phrase-Level Recovery

- On error discovery, parser may perform local correction on the remaining input.
- It may replace a prefix of the remaining input by some string that allows the parser to continue.
- Correction maybe to replace a comma by a semicolon.

Error Productions

- Augment the grammar for the language with productions that generate erroneous constructs.
- Use the modified grammar to construct a parser.

Global Correction

- Compiler is expected to make as few changes as possible in processing an incorrect input string.
- Given an incorrect input string x and grammar G , algorithms will find a parse tree for a related string y such that the number of insertions, deletions, and changes of tokens required to transform x into y is as small as possible.

2.1 SYNTAX ANALYSIS

Syntax analysis or parsing is the second phase of a compiler.

A lexical analyzer can identify tokens with the help of regular expressions and pattern rules. But a lexical analyzer cannot check the syntax of a given sentence due to the limitations of the regular expressions.

Regular expressions cannot check balancing tokens, such as parenthesis. Therefore, this phase uses context-free grammar (CFG), which is recognized by push-down automata.

2.1.1 REVIEW OF CONTEXT FREE GRAMMARS

A context-free grammar (grammar for short) consists of terminals, nonterminals, a start symbol, and productions.

1. **Terminals** are the basic symbols from which strings are formed. The term "token name" is a synonym for "terminal" and frequently we will use the word "token" for terminal when it is clear that we are talking about just the token name.
2. **Nonterminals** are syntactic variables that denote sets of strings. The nonterminals define sets of strings that help define the language generated by the grammar. They also impose a hierarchical structure on the language that is useful for Both syntax analysis and translation.
3. In a grammar, one nonterminal is distinguished as the **start symbol**, and the set of strings it denotes is the language generated by the grammar. Conventionally, the productions for the start symbol are listed first.
4. The **productions** of a grammar specify the manner in which the terminals and nonterminals can be combined to form strings. Each production consists of:
 - a. A nonterminal called the head or left side of the production; this production defines some of the strings denoted by the head.
 - b. The symbol $\rightarrow$. Sometimes $::=$ has been used in place of the arrow.
 - c. A body or right side consisting of zero or more terminals and nonterminals.

EXAMPLE:

The grammar with the following productions defines simple arithmetic expression:

```
expression → expression + term
expression → expression - term
expression → term
term        → term * factor
term        → term / factor
term        → factor
factor      → ( expression )
factor      → id
```

In this grammar, the **terminal symbols** are : id + - * / ()

The **nonterminal symbols** are : expression, term, factor

Start symbol : expression

Notational Conventions

To avoid always having to state that "these are the terminals," "these are the nonterminals," and so on, the following notational conventions for grammars will be used.

✚ These symbols are terminals:

- Lowercase letters early in the alphabet, such as a, b, c.
- Operator symbols such as +, *, and so on.
- Punctuation symbols such as parentheses, comma, and so on.
- The digits 0, 1, . . . , 9.
- Boldface strings such as id or if, each of which represents a single terminal symbol.

✚ These symbols are nonterminals:

- Uppercase letters early in the alphabet, such as A, B, C.
- The letter S, which, when it appears, is usually the start symbol.
- Lowercase, italic names such as expr or stmt.
- When discussing programming constructs, uppercase letters may be used to represent nonterminals for the constructs. For example, nonterminals for expressions, terms, and factors are often represented by E, T, and F, respectively.

✚ Uppercase letters late in the alphabet, such as X, Y, Z, represent grammar symbols; that is, either nonterminals or terminals.

✚ Lowercase letters late in the alphabet, chiefly u, v, ... ,z, represent (possibly empty) strings of terminals.

✚ Lowercase Greek letters α , β , γ for example, represent (possibly empty) strings of grammar symbols.

✚ A set of productions $A \rightarrow \alpha_1, A \rightarrow \alpha_2, \dots, A \rightarrow \alpha_k$ with a common head A (call them A-productions), may be written $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_k$. We call $\alpha_1, \alpha_2, \dots, \alpha_k$ the alternatives for A.

✚ Unless stated otherwise, the head of the first production is the start symbol.

Using these conventions, the grammar for arithmetic expression can be rewritten as:

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

2.1.1.1 DERIVATION TREES AND PARSE TREES

The construction of a parse tree can be made precise by taking a derivational view, in which productions are treated as rewriting rules.

Beginning with the start symbol, each rewriting step replaces a nonterminal by the body of one of its productions.

For example, consider the following grammar, with a single nonterminal E:

$$E \rightarrow E + E \mid E * E \mid - E \mid (E) \mid id$$

The production $E \rightarrow - E$ signifies that if E denotes an expression, then $- E$ must also denote an expression. The replacement of a single E by $- E$ will be described by writing $E \Rightarrow -E$ which is read, "E derives $- E$."

The production $E \rightarrow (E)$ can be applied to replace any instance of E in any string of grammar symbols by (E), e.g., $E * E \Rightarrow (E) * E$ or $E * E \Rightarrow E * (E)$

We can take a single E and repeatedly apply productions in any order to get a sequence of replacements. For example, $E \Rightarrow - E \Rightarrow - (E) \Rightarrow - (id)$

We call such a sequence of replacements a derivation of $-(id)$ from E. This derivation provides a proof that the string $-(id)$ is one particular instance of an expression.

Derivation Order

- | | | |
|-----------------------|----------------------------|----------------------------|
| 1. $S \rightarrow AB$ | 2. $A \rightarrow aaA$ | 4. $B \rightarrow Bb$ |
| | 3. $A \rightarrow \lambda$ | 5. $B \rightarrow \lambda$ |

Leftmost derivation:

$$S \xRightarrow{1} AB \xRightarrow{2} aaAB \xRightarrow{3} aaB \xRightarrow{4} aaBb \xRightarrow{5} aab$$

Rightmost derivation:

$$S \xRightarrow{1} AB \xRightarrow{4} ABb \xRightarrow{5} Ab \xRightarrow{2} aaAb \xRightarrow{3} aab$$

Leftmost And Rightmost Derivation Of A String

-  **Leftmost derivation** – A leftmost derivation is obtained by applying production to the leftmost variable in each step.
-  **Rightmost derivation** – A rightmost derivation is obtained by applying production to the rightmost variable in each step.

Example

Let any set of production rules in a CFG be

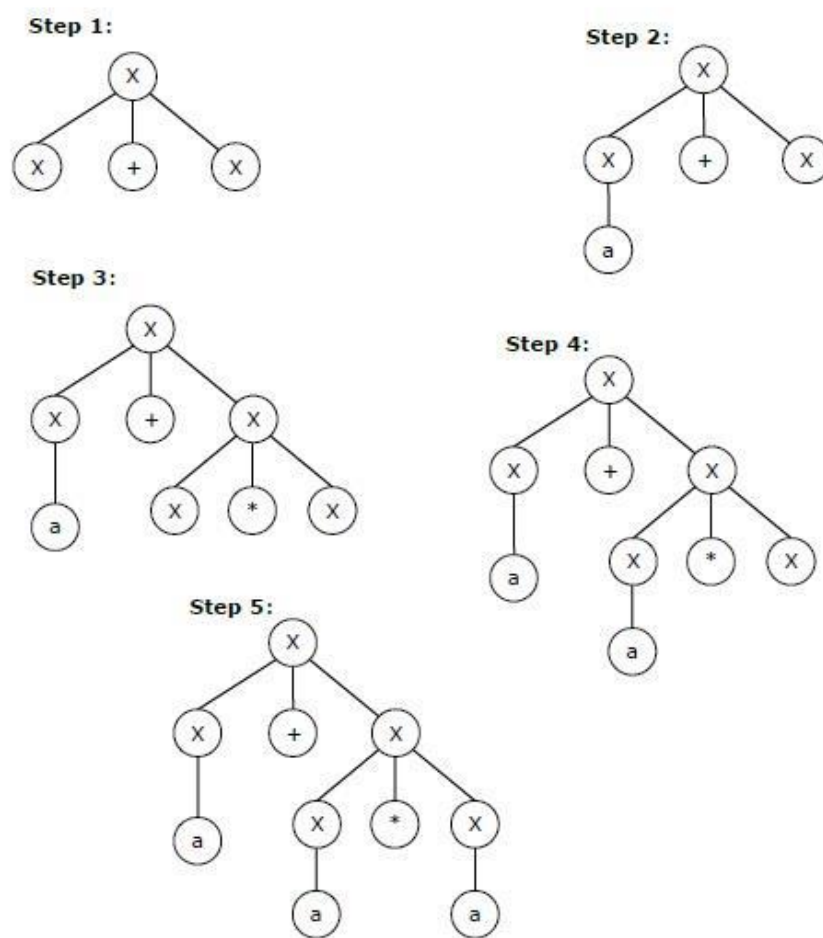
$$X \rightarrow X+X \mid X*X \mid X \mid a$$

over an alphabet $\{a\}$.

The leftmost derivation for the string "a+a*a" may be –

$$X \rightarrow X+X \rightarrow a+X \rightarrow a + X*X \rightarrow a+a*X \rightarrow a+a*a$$

The stepwise derivation of the above string is shown as below –



The rightmost derivation for the above string "a+a*a" may be

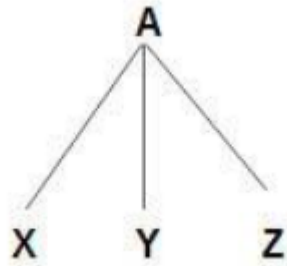
$$X \rightarrow X*X \rightarrow X*a \rightarrow X+X*a \rightarrow X+a*a \rightarrow a+a*a$$

Parse Tree

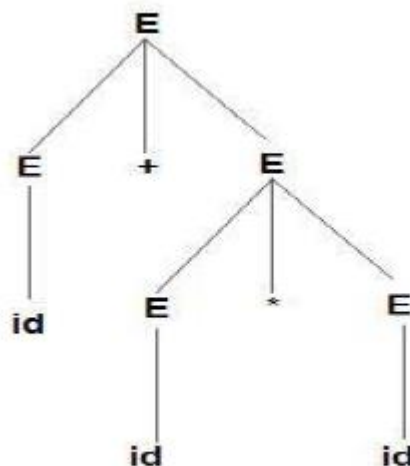
Parse tree is a hierarchical structure which represents the derivation of the grammar to yield input strings.

Simply it is the graphical representation of derivations.

- ✚ Root node of parse tree has the start symbol of the given grammar from where the derivation proceeds.
- ✚ Leaves of parse tree are labeled by non-terminals or terminals.
- ✚ Each interior node is labeled by some non terminals.
- ✚ If $A \rightarrow xyz$ is a production, then the parse tree will have **A** as interior node whose children are **x**, **y** and **z** from its left to right.



- ✚ Construct parse tree for $E \rightarrow E + E / E * E / id$



Yield Of Parse Tree

- ✚ The leaves of the parse tree are labeled by nanterminals or terminals and read from left to right, they constitute a sentential form, called the yield or frontier of the tree.
- ✚ Figure above represents the parse tree for the string **id+ id*id**. The string **id + id * id**, is the yield of parse tree depicted in Figure.

2.1.1.2 AMBIGUITY

- ✚ An ambiguous grammar is one that produces more than one leftmost or more than one rightmost derivation for the same sentence.
- Or in other words, a Grammar is ambiguous if it produces more than one parse tree for some sentence. A language generated by an ambiguous grammar is said to be inherently ambiguous. Ambiguity in grammar is not good for compiler construction. No method can

Removal of Ambiguity:

Ambiguity can be removed on the basis of the following two properties –

1. Precedence - If different operators are used, the precedence of the operators are considered.

- The level at which the production is present denotes the priority of the operator used.
- The production at **higher levels** will have **operators with less priority**. In the parse tree, the nodes which are at top levels or close to the root node will contain the lower priority operators.
- The production at **lower levels** will have operators with **higher priority**. In the parse tree, the nodes which are at lower levels or close to the leaf nodes will contain the higher priority operators.

2. Associativity - If the same precedence operators are in production then associativity is considered.

- If the associativity is left to right, then we have to prompt a left recursion in the production. The parse tree will also be left recursive and grow on the left side. +, -, *, / are left associative operators.
- If the associativity is right to left, then we have to prompt the right recursion in the productions. The parse tree will also be right recursive and grow on the right side. ^ is a right associative operator.

So by considering the above rules, the grammar $E \rightarrow E + E \mid E * E \mid id$ can be replaced by the following grammar which is ambiguous.

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow id$

Example:

Convert the following grammar to unambiguous grammar

$E \rightarrow E * E \mid E - E$
 $E \rightarrow E ^ E \mid E / E$
 $E \rightarrow E + E$
 $E \rightarrow (E) \mid id$

Solution:

The grammar can be converted to unambiguous grammar using the precedence of operators as well as associativity.

Step1: Arrange the operators in increasing order of the precedence along with the associativity as shown below:

Operators	Associativity	Non Terminal used
+, -	Left-Associative	E
*, /	Left-Associative	T
^	Right-Associative	P

Terminals: (), id F

The final grammar which is unambiguous is given below:

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow T * P \mid T / P \mid P$

$P \rightarrow F \wedge P \mid F$

$F \rightarrow (E) \mid id$

detect and remove ambiguity automatically, but it can be removed either by rewriting the whole grammar without ambiguity or by setting and following associativity and precedence constraints.

- For most parsers, it is desirable that the grammar be made unambiguous, for if it is not, we cannot uniquely determine which parse tree to select for a sentence.
- In other cases, it is convenient to use carefully chosen ambiguous grammars, together with disambiguating rules that "throw away" undesirable parse trees, leaving only one tree for each sentence.

EXAMPLE

Consider very simple sentence $\text{id} + \text{id} * \text{id}$.

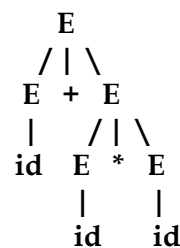
1st Leftmost Derivation

$$\begin{aligned} E &\Rightarrow E + E \\ &\Rightarrow \text{id} + E \\ &\Rightarrow \text{id} + E * E \\ &\Rightarrow \text{id} + \text{id} * E \\ &\Rightarrow \text{id} + \text{id} * \text{id} \end{aligned}$$

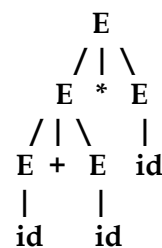
2nd Leftmost Derivation

$$\begin{aligned} E &\Rightarrow E * E \\ &\Rightarrow E + E * E \\ &\Rightarrow \text{id} + \text{id} * E \\ &\Rightarrow \text{id} + \text{id} * E \\ &\Rightarrow \text{id} + \text{id} * \text{id} \end{aligned}$$

1st Parse Tree



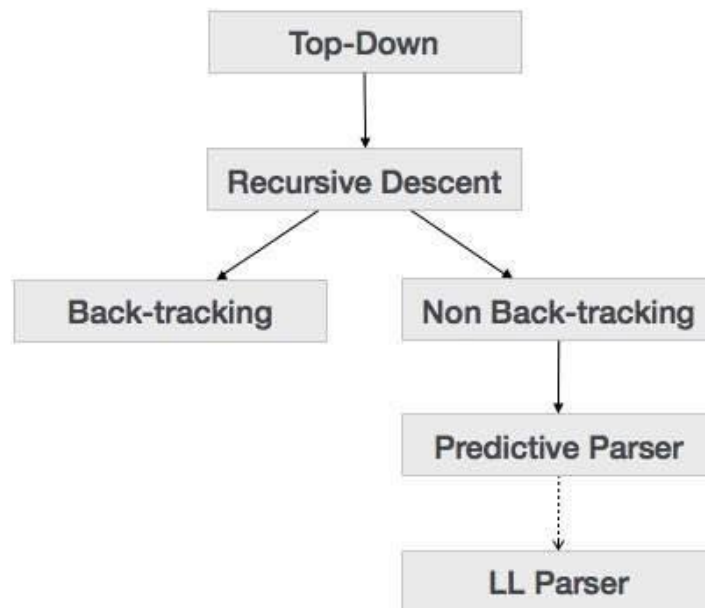
2nd Parse Tree



2.2 TOP DOWN PARSING

- Parsing is the process of determining if a string of token can be generated by a grammar.
- Mainly 2 parsing approaches:
 - **Top Down Parsing**
 - **Bottom Up Parsing**
- In **top down parsing**, parse tree is constructed from top (root) to the bottom (leaves).
- In **bottom up parsing**, parse tree is constructed from bottom (leaves) to the top (root).
- It can be viewed as an attempt to construct a parse tree for the input starting from the root and creating the nodes of parse tree in preorder.
- Pre-order traversal means: 1. Visit the root 2. Traverse left subtree 3. Traverse right subtree.

- ✚ Top down parsing can be viewed as an attempt to find a leftmost derivation for an input string (that is expanding the leftmost terminal at every step).



2.2.1 RECURSIVE DESCENT PARSING

It is the most general form of top-down parsing.

It may involve **backtracking**, that is making repeated scans of input, to obtain the correct expansion of the leftmost non-terminal. Unless the grammar is ambiguous or left-recursive, it finds a suitable parse tree

EXAMPLE

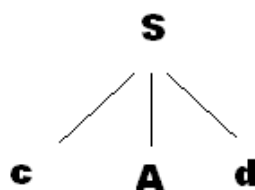
Consider the grammar:

$S \rightarrow cAd$

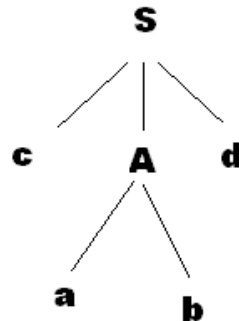
$A \rightarrow ab \mid a$

and the input string $w = cad$.

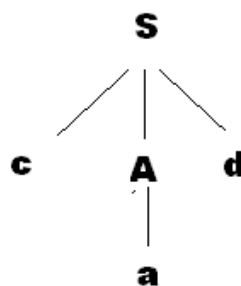
- ❖ To construct a parse tree for this string top down, we initially create a tree consisting of a single node labelled **S**.
- ❖ An input pointer points to **c**, the first symbol of w . **S** has only one production, so we use it to expand **S** and obtain the tree as:



- ❖ The leftmost leaf, labeled **c**, matches the first symbol of input **w**, so we advance the input pointer to **a**, the second symbol of **w**, and consider the next leaf, labeled **A**.
- ❖ Now, we expand **A** using the first alternative $A \rightarrow ab$ to obtain the tree as:



- ❖ We have a match for the second input symbol, **a**, so we advance the input pointer to **d**, the third input symbol, and compare **d** against the next leaf, labeled **b**.
- ❖ Since **b** does not match **d**, we report failure and go back to **A** to see whether there is another alternative for **A** that has not been tried, but that might produce a match.
- ❖ In going back to **A**, we must reset the input pointer to position 2, the position it had when we first came to **A**, which means that the procedure for **A** must store the input pointer in a local variable.
- ❖ The second alternative for **A** produces the tree as:



- ❖ The leaf **a** matches the second symbol of **w** and the leaf **d** matches the third symbol. Since we have produced a parse tree for **w**, we halt and announce successful completion of parsing. (that is the string parsed completely and the parser stops).
- ❖ The leaf **a** matches the second symbol of **w** and the leaf **d** matches the third symbol. Since we have produced a parse tree for **w**, we halt and announce successful completion of parsing. (that is the string parsed completely and the parser stops).

2.2.2 PREDICTIVE PARSING

- ✚ A predictive parsing is a special form of recursive-descent parsing, in which the current input token unambiguously determines the production to be applied at each

step. The goal of predictive parsing is to construct a top-down parser that never backtracks. To do so, we must transform a grammar in two ways:

- **Eliminate left recursion, and**
- **Perform left factoring.**

✚ These rules eliminate most common causes for backtracking although they do not guarantee a completely backtrack-free parsing (called LL(1) as we will see later).

Left Recursion

✚ A grammar is said to be left -recursive if it has a non-terminal A such that there is a derivation $A \rightarrow A\alpha$, for some string α .

EXAMPLE

Consider the grammar

$$A \rightarrow A\alpha$$

$$A \rightarrow \beta$$

- ❖ It recognizes the regular expression $\beta\alpha^*$. The problem is that if we use the first production for top-down derivation, we will fall into an infinite derivation chain. This is called left recursion.
- ❖ Top-down parsing methods cannot handle left recursive grammars, so a transformation that eliminates left-recursion is needed. The left-recursive pair of productions $A \rightarrow A\alpha | \beta$ could be replaced by two non-recursive productions.

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' | \epsilon$$

✚ Consider The following grammar which generates arithmetic expressions

$$E \rightarrow E + T | T$$

$$T \rightarrow T * F | F$$

$$F \rightarrow (E) | id$$

Eliminating the immediate left recursion to the productions for E and then for T, we obtain

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' | \epsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' | \epsilon$$

$$F \rightarrow (E) | id$$

- ✚ No matter how many A-productions there are, we can eliminate immediate left recursion from them by the following technique. First, we group the A productions as

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

where no β_i begins with an A. Then we replace the A-productions by

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \varepsilon$$

Left Factoring

- ✚ Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive parsing.
- ✚ The basic idea is that when it is not clear which of two alternative productions to use to expand a non-terminal A, we may be able to rewrite the A-productions to defer the decision until we have seen enough of the input to make the right choice

$$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$$

- ✚ $\alpha \beta_1$ and $\alpha \beta_2$ are two A-productions, and the input begins with a non-empty string derived from α we do not know whether to expand A to $\alpha \beta_1$ or $\alpha \beta_2$.
- ✚ However, we may defer the decision by expanding A to αB . Then, after seeing the input derived from α , we may expand B to β_1 or β_2 .
- ✚ The left factored original expression becomes:

$$\begin{aligned} A &\rightarrow \alpha B \\ B &\rightarrow \beta_1 \mid \beta_2 \end{aligned}$$

- ✚ For the “dangling else” grammar:

$$\text{stmt} \rightarrow \text{if cond then stmt else stmt} \mid \text{if cond then stmt}$$

The corresponding left – factored grammar is:

$$\text{stmt} \rightarrow \text{if cond then stmt else_clause}$$

$$\text{else_clause} \rightarrow \text{else stmt} \mid \varepsilon$$

Non Recursive Predictive parser

- ✚ It is possible to build a nonrecursive predictive parser by maintaining a stack explicitly, rather than implicitly via recursive calls.
- ✚ The key problem during predictive parsing is that of determining the production to be applied for a nonterminal.
- ✚ The nonrecursive parser in looks up the production to be applied in a parsing table

Requirements

1. Stackv
2. Parsing Table
3. Input Buffer
4. Parsing

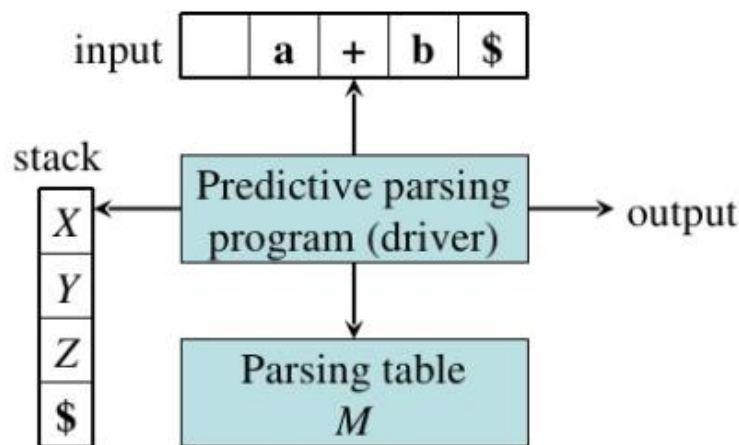


Figure : Model of a nonrecursive predictive parser

- ✚ **Input buffer** - contains the string to be parsed, followed by \$(used to indicate end of input string)
- ✚ **Stack** - initialized with \$, to indicate bottom of stack.
- ✚ **Parsing table** - 2 D array $M[A, a]$ where A is a nonterminal and a is terminal or the symbol \$
- ✚ The parser is controlled by a program that behaves as follows. The program considers X , the symbol on top of the stack, and a current input symbol. These two symbols determine the action of the parser.
- ✚ There are three possibilities,
 1. If $X = a = \$$, the parser halts and announces successful completion of parsing.
 2. If $X = a \neq \$$, the parser pops X off the stack and advances the input pointer to the next input symbol,
 3. If X is a nonterminal, the program consults entry $M[X, a]$ of the parsing table M . The entry will be either an X -production of the grammar or an error entry. If, for example, $M[X, u] = \{X \rightarrow UVW\}$, the parser replaces X on top of the stack by WVU (with U on top). As output we shall assume that the parser just prints the production used; any other code could be executed here. If $M[X, a] = \text{error}$, the parser calls an error recovery routine.

Predictive Parsing Algorithm

INPUT: A string w and a parsing table M for grammar G .

OUTPUT: If w is in $L(G)$, a leftmost derivation of w ; otherwise, an error indication.

METHOD: Initially, the parser is in a configuration in which it has $\$S$ on the stack with S , the start symbol of G on top, and $w\$$ in the input buffer. The program that utilizes the predictive parsing table M to produce a parse for the input is shown below.

set ip to point to the first symbol of $w\$$;

repeat

 let X be the top stack symbol and a the symbol pointed to by ip;

 if X is a terminal or $\$$ then

 if $X = a$ then

 pop X from the stack and advance ip

 else error ()

 else /* X is a nonterminal */

 if $M[X, a] = X \rightarrow Y_1 Y_2 \dots Y_i$ then begin

 pop X from the stack;

 push $Y_k, Y_{k-1}, \dots, Y_1$ onto the stack, with Y_1 on top;

 output the production $X \rightarrow Y_1 Y_2 \dots Y_k$

 end

 else error ()

until $X = \$$ /* stack is empty */

Construction Of Predictive Parsing Table

✚ Uses 2 functions:

➤ **FIRST()**

➤ **FOLLOW()**

✚ These functions allows us to fill the entries of predictive parsing table

FIRST

✚ If ' α ' is any string of grammar symbols, then $FIRST(\alpha)$ be the set of terminals that begin the string derived from α . If $\alpha \Rightarrow^* \epsilon$ then add ϵ to $FIRST(\alpha)$. First is defined for both terminals and non terminals.

✚ To Compute First Set

1. If X is a terminal, then $FIRST(X)$ is $\{X\}$
2. If $X \rightarrow \epsilon$ then add ϵ to $FIRST(X)$
3. If X is a non terminal and $X \rightarrow Y_1 Y_2 Y_3 \dots Y_n$, then put 'a' in $FIRST(X)$ if for some i , a is in $FIRST(Y_i)$ and ϵ is in all of $FIRST(Y_1), \dots, FIRST(Y_{i-1})$.

EXAMPLE

Consider Grammar:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

$F \rightarrow (E) \mid id$

Non-terminal	FIRST
E	(, id
E'	+, ϵ
T	(, id
T'	*, ϵ
F	(, id

FOLLOW

- ✚ FOLLOW is defined only for non-terminals of the grammar G .
- ✚ It can be defined as the set of terminals of grammar G , which can immediately follow the non-terminal in a production rule from start symbol.
- ✚ In other words, if A is a nonterminal, then $\text{FOLLOW}(A)$ is the set of terminals 'a' that can appear immediately to the right of A in some sentential form.

✚ Rules to Compute Follow Set

1. If S is the start symbol, then add $\$$ to the $\text{FOLLOW}(S)$.
2. If there is a production rule $A \rightarrow \alpha B \beta$ then everything in $\text{FIRST}(\beta)$ except for ϵ is placed in $\text{FOLLOW}(B)$.
3. If there is a production $A \rightarrow \alpha B$, or a production $A \rightarrow \alpha B \beta$ where $\text{FIRST}(\beta)$ contains ϵ then everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.

EXAMPLE

Consider Grammar:

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \epsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' \mid \epsilon$$

$$F \rightarrow (E) \mid \text{id}$$

Non-terminal	FIRST	FOLLOW
E	(, id	), \$
E'	+, ϵ	), \$
T	(, id	+,), \$
T'	*, ϵ	+,), \$
F	(, id	+, *,), \$

EXAMPLE

1. $S \rightarrow A a$
2. $A \rightarrow B D$
3. $B \rightarrow b$
4. $B \rightarrow \epsilon$
5. $D \rightarrow d$
6. $D \rightarrow \epsilon$

$$\text{First}(S) = \{b, d, a\}$$

$$\text{First}(A) = \{b, d, \epsilon\}$$

$$\text{First}(B) = \{b, \epsilon\}$$

$$\text{First}(D) = \{d, \epsilon\}$$

$$\text{Follow}(S) = \{\$ \}$$

$$\text{Follow}(A) = \{a\}$$

$$\text{Follow}(B) = \{d, a\}$$

$$\text{Follow}(D) = \{a\}$$

EXAMPLE

$S \rightarrow A, A \rightarrow BC \mid DBC, B \rightarrow b B' \mid \epsilon, B' \rightarrow b B' \mid \epsilon, C \rightarrow c \mid \epsilon, D \rightarrow a \mid d$

Non-terminal	first	follow
S	$\{b, c, a, d, \epsilon\}$	$\{\$ \}$
A	$\{b, c, a, d, \epsilon\}$	$\{\$ \}$
B	$\{b, \epsilon\}$	$\{c, \$ \}$
B'	$\{b, \epsilon\}$	$\{c, \$ \}$
C	$\{c, \epsilon\}$	$\{\$ \}$
D	$\{a, d\}$	$\{b, c, \$ \}$

Algorithm To Construct A Predictive Parsing Table.

INPUT: Grammar G .

OUTPUT: Parsing table M .

METHOD

1. For each production $A \rightarrow \alpha$ of the grammar, do steps 2 and 3.
2. For each terminal a in $\text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ to $M | A, a |$
3. If ϵ is in $\text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ to $M | A, b |$ for each terminal b in $\text{FOLLOW}(A)$. If ϵ is in $\text{FIRST}(\alpha)$ and $\$$ is in $\text{FOLLOW}(A)$, add $A \rightarrow \alpha$ to $M | A, \$ |$
4. Make each undefined entry of M be error

Non-terminal	FIRST	FOLLOW
E	(, id	), \$
E'	+, ϵ	), \$
T	(, id	+,), \$
T'	*, ϵ	+,), \$
F	(, id	+, *,), \$

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Parsing Table

Blank entries are error states. For example, E cannot derive a string starting with '+'

STACK	INPUT	OUTPUT
\$E	id + id * id\$	
\$E' T	id + id * id\$	$E \rightarrow T E'$
\$E' T' F	id + id * id\$	$T \rightarrow F T'$
\$E' T' id	id + id * id\$	$F \rightarrow id$
\$E' T'	+ id * id\$	
\$E'	+ id * id\$	$T' \rightarrow \epsilon$
\$E' T +	+ id * id\$	$E' \rightarrow + T E'$
\$E' T	id * id\$	
\$E' T' F	id * id\$	$T \rightarrow F T'$
\$E' T' id	id * id\$	$F \rightarrow id$
\$E' T'	* id\$	
\$E' T' F *	* id\$	$T' \rightarrow * F T'$
\$E' T' F	id\$	
\$E' T' id	id\$	$F \rightarrow id$
\$E' T'	\$	
\$E'	\$	$T' \rightarrow \epsilon$
\$	\$	$E' \rightarrow \epsilon$

Moves made by predictive parser for the input **id+id*id**

2.2.3 LL(1) GRAMMERS

- ✚ LL(1) grammars are the class of grammars from Which the predictive parsers can be constructed automatically.
- ✚ A context-free grammar $G = (VT, VN, P, S)$ whose parsing table has no multiple entries is said to be LL(1).
- ✚ In the name LL(1),
 - the first L stands for scanning the input from left to right,
 - the second L stands for producing a leftmost derivation,
 - and the 1 stands for using one input symbol of lookahead at each step to make parsing action decision.
- ✚ A language is said to be LL(1) if it can be generated by a LL(1) grammar. It can be shown that LL(1) grammars are
 - not ambiguous and

- not left-recursive

EXAMPLE

Consider the following grammar

$$S \rightarrow i E t S S' \mid a$$

$$S' \rightarrow e S \mid \epsilon$$

$$E \rightarrow b$$

Non-terminal	FIRST	FOLLOW
S	i, a	e, \$
S'	e, ϵ	e, \$
E	b	t

Non-terminal	i	t	a	e	b	\$
S	$S \rightarrow i E t S S'$		$S \rightarrow a$			
S'				$S' \rightarrow e S$ $S' \rightarrow \epsilon$		$S' \rightarrow \epsilon$
E					$E \rightarrow b$	

This is not LL(1) grammar
